

0.81.1. Ramasser une pièce : le P — elle réapparaît ailleurs

Le 0.81, mais la pièce ramassée réapparaît à la place suivante d'un tableau : on peut en ramasser sans fin.

1. Ce qu'il faut comprendre

C'est le 0.81, mais au lieu de partir hors de l'écran, la pièce **réapparaît ailleurs**.

Ce qui est nouveau ici : des places rangées dans deux tableaux, `PX` et `PY` (comme les pas du 0.32). La pièce n° `k` est en (`PX[k]`, `PY[k]`). Ramassée, on passe à la suivante : `k = (k + 1) % 5;` — après la 5e, on revient à la 1re.

On la repose aussitôt à sa nouvelle place, avec `poser`.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction `poser\(\)` — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <texte>` → [0.0.2. La fonction `texte\(\)` — écrire un mot](#)
- `#include <deplace_croix>` → [0.66.3. La fonction `deplace\_croix\(\)` — suivre la croix, case par case](#)
- `#include <nombre>` → [0.8.3. La fonction `nombre\(\)` — écrire un nombre](#)
- `#include <reste>` → [0.8.4. `#include <reste>` — `a % b`](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1372 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage      87 o ← la console, à l'allumage - la mise en route de la console
deplace_croix() 217 o ← main() - une fonction fournie par le langage
main()         141 o ← démarrage - ta fonction
VBlank         8 o  ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EffacerCases, EcrireTexte, EcrireNombre - attend de pouvoir écrire une case
AdresseCase    10 o ← effacer(), poser() - trouve l'adresse d'une case de l'écran
EffacerCases    12 o ← effacer() - efface des cases
EcrireTexte     10 o ← texte() - écrit un texte, lettre par lettre
EcrireNombre    33 o ← nombre() - écrit un nombre en chiffres
Reste          10 o ← main() - le reste d'une division
CopierTuiles    18 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte    14 o ← démarrage - vide les deux cartes de l'écran
LireManette     37 o ← bouton() - lit les huit boutons
Tuiles         704 o ← CopierTuiles - la police entière
"SCORE"        5 o  ← texte() - un texte gravé
```

```

PX_0          5 o ← main() – une table gravée
PY_1          5 o ← main() – une table gravée
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, Diviser, Multiplier, DecalerGauche, DecalerDroite,
RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes
Fonctions du langage appelées :
  poser()      × 3    39 o sur place, dans deplace_croix(), main() + AdresseCase (10 o)
  bouton()     × 4    20 o sur place, dans deplace_croix() + LireManette (37 o)
  effacer()    × 1    15 o sur place, dans deplace_croix() + AdresseCase (10 o), EffacerCases (12 o)
  texte()     × 1    11 o sur place, dans main() + EcrireTexte (10 o)
  nombre()    × 1    10 o sur place, dans main() + EcrireNombre (33 o)
  image()     × 1     3 o sur place, dans main() + AttendreImage (28 o)
  images()    × 1     2 o sur place, dans deplace_croix()
Tes fonctions appelées :
  deplace_croix() × 1    19 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// ---- LES VARIABLES : tout ce que le jeu doit retenir ----

#include <poser>          // pose une tuile sur une case du fond
#include <ALPHABET>       // les lettres de la police : ALPHABET[0] est le A
#include <texte>          // écrit un texte à l'écran
#include <deplace_croix>  // une tuile qui suit la croix, case par case
#include <nombre>         // écrit un nombre en chiffres
#include <reste>          // a % b, sauf par 1, 2, 4, 8, 16... écrits en clair

```

2. UNE VARIABLE — X

```

uint8_t x = 5;          // la COLONNE du A (0 à gauche ... 19 à droite) : il part de la colonne 5

```

3. UNE VARIABLE — Y

```

uint8_t y = 8;          // la LIGNE du A (0 en haut ... 17 en bas) : il part de la ligne 8

```

4. UNE VARIABLE — PX

```

uint8_t px = 15;        // la colonne de la PIÈCE. La pièce est une LETTRE : le P.

```

5. UNE VARIABLE — PY

```

uint8_t py = 8;         // sa ligne. Elle est donc en (15, 8), sur la même ligne que le A.

```

6. UNE VARIABLE — SCORE

```

uint8_t score = 0;      // combien de pièces on a ramassées : 0 au départ

// Les places des pièces, l'une après l'autre, dans deux tableaux (le 0.32) :
//
//  pièce n° :      0   1   2   3   4
//  PX (colonne): 15,  3, 17,  8, 12
//  PY (ligne) :   8,  2, 14,  5, 11
//
//  la pièce n° 0 est en (15, 8), la n° 1 en (3, 2), la n° 2 en (17, 14)...

```

7. UNE TABLE GRAVÉE DANS LA CARTOUCHE — PX

```

const uint8_t PX[] = { 15,  3, 17,  8, 12 };

```

8. UNE TABLE GRAVÉE DANS LA CARTOUCHE — PY

```

const uint8_t PY[] = {  8,  2, 14,  5, 11 };

```

9. UNE VARIABLE — K

```
uint8_t k = 0;           // le numéro de la pièce EN COURS : 0, 1, 2, 3, 4, puis de nouveau 0
```

10. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    poser(px, py, ALPHABET[15]);
    texte(0, 17, "SCORE");

    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 250);

        if (x == px && y == py) {           // le A est sur la pièce (le 0.81)
            score = score + 1;              // une pièce de plus

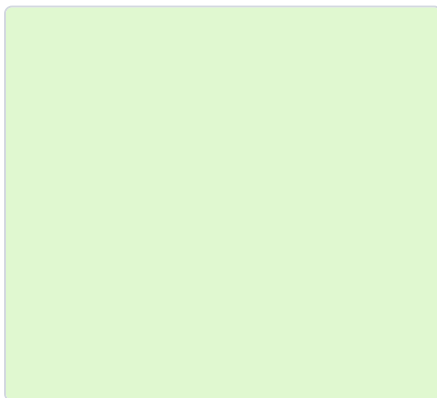
            // La pièce SUIVANTE du tableau :
            //
            //   k = (k + 1) % 5;
            //   |           |
            //   |           +-- % 5 : le reste de la division par 5. 5 % 5 = 0 :
            //   |           après la pièce n° 4, on revient à la n° 0.
            //   +----- k + 1 : le numéro suivant
            //
            //   k : 0 → 1 → 2 → 3 → 4 → 0 → 1 ... sans fin
            k = (k + 1) % 5;

            px = PX[k];                     // la colonne de la pièce n° k...
            py = PY[k];                     // ...et sa ligne
            poser(px, py, ALPHABET[15]);    // on y pose le P : la nouvelle pièce apparaît
        }

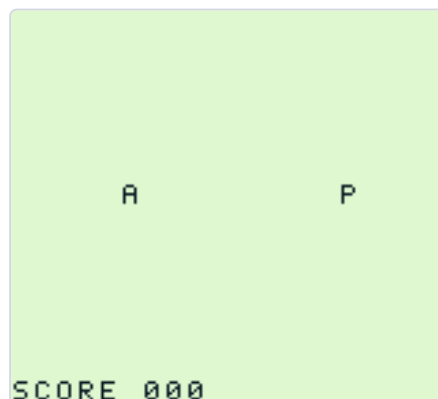
        nombre(6, 17, score);
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Le A ramasse le P ; un autre P apparaît aussitôt ailleurs, et ainsi de suite.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ la première pièce est ramassée

✓ une nouvelle pièce en (3, 2)

Ensuite : **0.81.2. La fonction `hasard()` — un nombre au hasard** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++